



Arbre Binaire de Recherche Rouge-Noir

Parcours, Recherche et Amelioration de l'equilibrage

Etudiant	Sojarilala Andrianantenaina Francois de Paul
Niveau	Licence 1 - Informatique
Etablissement	INSI - Antananarivo, Madagascar
Langage	Java (JDK 21) + JavaFX pour la visualisation
Sujet	Parcours et recherche (prefixe, infixe, postfixe) + amelioration : Arbre Rouge-Noir

1. Introduction

Ce projet a pour objectif d'implémenter, sans utiliser de bibliothèque de structure de données, un Arbre Binaire de Recherche (ABR) équilibré de type **Rouge-Noir**. L'exercice classique de parcours et de recherche sur un ABR y est étendu par un mécanisme d'auto-équilibrage, qui garantit que la hauteur de l'arbre reste en **$O(\log n)$** quel que soit l'ordre d'insertion des clés — contrairement à un ABR simple qui peut dégénérer en liste chaînée dans le pire cas (insertion de clés déjà triées, par exemple).

L'implémentation respecte la contrainte de n'utiliser **aucune bibliothèque de structure de données** (pas de TreeMap, LinkedList, Stack, etc.) : l'arbre est construit entièrement à partir d'une classe Node et de pointeurs manuels (gauche, droite, parent), à la manière de l'algorithme présenté dans Cormen, Leiserson, Rivest & Stein (CLRS), référence académique standard pour cette structure.

1.1 Objectifs du projet

- Implémenter l'insertion et la suppression avec re-équilibrage automatique (rotations et recoloration).
- Implémenter la recherche d'une clé en $O(\log n)$.
- Implémenter les trois parcours classiques : préfixe, infixe et postfixe.
- Vérifier automatiquement, après chaque opération, que les 5 propriétés Rouge-Noir sont respectées.
- Fournir une interface graphique (JavaFX) pour visualiser l'arbre et son équilibrage en temps réel.

1.2 Pourquoi un arbre Rouge-Noir ?

Un ABR classique n'offre de bonnes performances ($O(\log n)$) que si les insertions se font dans un ordre suffisamment aléatoire. Dans le pire cas (insertions triées), sa hauteur devient $O(n)$, dégradant la recherche au même coût qu'une liste chaînée. L'arbre Rouge-Noir résout ce problème en imposant des contraintes de coloration qui bornent la hauteur à $2 \times \log_2(n+1)$ au maximum, quelle que soit la séquence d'insertions.

2. Les proprietes de l'arbre Rouge-Noir

Un arbre Rouge-Noir est un ABR dans lequel chaque noeud possede une couleur (**ROUGE** ou **NOIR**), et qui respecte les cinq proprietes suivantes :

1	Chaque noeud est soit ROUGE, soit NOIR.
2	La racine est toujours NOIRE.
3	Chaque feuille (pointeur NIL) est consideree NOIRE.
4	Un noeud ROUGE ne peut pas avoir d'enfant ROUGE (pas de rouge-rouge consecutif).
5	Tout chemin d'un noeud vers ses feuilles NIL descendantes contient le meme nombre de noeuds NOIRS (haut

Ces cinq proprietes, combinees, garantissent que le chemin le plus long depuis la racine n'est jamais plus de deux fois plus long que le chemin le plus court : l'arbre est donc **approximativement equilibre**. C'est cette garantie qui assure une complexite $O(\log n)$ pour l'insertion, la suppression et la recherche.

2.1 Consequence : la hauteur est bornee

On peut demontrer que pour un arbre Rouge-Noir contenant n noeuds internes, la hauteur h verifie : $h \leq 2 \times \log_2(n + 1)$. Concretement, pour 1 million de noeuds, la hauteur ne depassera jamais environ 40, alors qu'un ABR non equilibre pourrait, dans le pire cas, atteindre une hauteur de 1 million.

3. Architecture du projet

Le projet est organise en quatre classes Java, toutes situees dans le meme package par default, sans dependance a une bibliotheque de structure de donnees :

Fichier	Role
Node.java	Noeud de l'arbre : cle (String), couleur, references gauche/droite/parent.
RedBlackTree.java	Coeur de l'algorithme : insertion, suppression, recherche, parcours, rotations, verification c
Main.java	Point d'entree console : jeu de tests demontrant toutes les operations.
RedBlackTreeApp.java	Interface graphique JavaFX : visualisation interactive de l'arbre.

3.1 La sentinelle NIL

A la maniere de CLRS, l'implementation utilise un unique noeud sentinelle NIL, toujours NOIR, qui represente toutes les feuilles de l'arbre. Cela simplifie grandement le code des rotations et du re-equilibrage, en evitant les verifications constantes de null.

4. Algorithme d'insertion

L'insertion se deroule en deux temps : (1) une insertion classique d'ABR par comparaison lexicographique des cle, le nouveau noeud etant colore ROUGE ; (2) une phase de **correction** (*fixup*) qui restaure les proprietes Rouge-Noir eventuellement violees par cette insertion.

4.1 Les trois cas de correction

Lorsqu'un noeud ROUGE *z* est insere sous un parent lui-meme ROUGE (violation de la propriete 4), trois cas se presentent selon la couleur de l'*oncle* de *z* (le frere de son parent) :

- **Cas 1 — l'oncle est ROUGE** : on recolore le parent et l'oncle en NOIR, le grand-parent en ROUGE, puis on remonte le probleme au grand-parent.
- **Cas 2 — l'oncle est NOIR, *z* est un enfant "en zigzag"** : une rotation ramene la situation au cas 3.
- **Cas 3 — l'oncle est NOIR, *z* est un enfant "aligne"** : une recoloration et une rotation simple resolvent definitivement le probleme.

A la fin de la boucle de correction, la racine est systematiquement repasee en NOIR (propriete 2).

4.2 Extrait de code — correction apres insertion

```
private void insererFixup(Node z) {
    while (z.parent.color == Node.ROUGE) {
        if (z.parent == z.parent.parent.left) {
            Node oncle = z.parent.parent.right;
            if (oncle.color == Node.ROUGE) {
                // Cas 1 : recoloration, on remonte
                z.parent.color = Node.NOIR;
                oncle.color = Node.NOIR;
                z.parent.parent.color = Node.ROUGE;
                z = z.parent.parent;
            } else {
                if (z == z.parent.right) {
                    z = z.parent;
                    leftRotate(z);          // Cas 2 -> ramene au cas 3
                }
                z.parent.color = Node.NOIR; // Cas 3
                z.parent.parent.color = Node.ROUGE;
                rightRotate(z.parent.parent);
            }
        } else { /* symetrique */ }
    }
    root.color = Node.NOIR;
}
```

Complexite : $O(\log n)$ — au plus $O(\log n)$ recolorations en remontant l'arbre, et au plus 2 rotations en fin de correction.

5. Algorithme de suppression

La suppression est l'opération la plus délicate de la structure. Trois situations sont possibles selon le nombre d'enfants du nœud à supprimer :

- Le nœud n'a pas d'enfant gauche : on le remplace directement par son enfant droit.
- Le nœud n'a pas d'enfant droit : on le remplace directement par son enfant gauche.
- Le nœud a deux enfants : on le remplace par son **successeur** (le minimum de son sous-arbre droit), qui, par définition, a au plus un enfant.

L'opération de reconnexion d'un sous-arbre à son parent est factorisée dans une méthode `transplant(u, v)`. Si le nœud physiquement retiré de l'arbre était NOIR, une unité de hauteur noire est perdue sur son chemin : une phase de correction `supprimerFixup` est alors nécessaire, avec quatre cas symétriques (selon la couleur du frère du nœud "double-noir" et de ses enfants).

5.1 Extrait de code — suppression

```
public boolean supprimer(String key) {
    Node z = rechercherNode(root, key);
    if (z == NIL) return false;

    Node y = z;
    boolean yColorOrigine = y.color;
    Node x;

    if (z.left == NIL) {
        x = z.right;
        transplant(z, z.right);
    } else if (z.right == NIL) {
        x = z.left;
        transplant(z, z.left);
    } else {
        y = minimum(z.right);           // successeur
        yColorOrigine = y.color;
        x = y.right;
        if (y.parent == z) {
            x.parent = y;
        } else {
            transplant(y, y.right);
            y.right = z.right;
            y.right.parent = y;
        }
        transplant(z, y);
        y.left = z.left;
        y.left.parent = y;
        y.color = z.color;
    }
    if (yColorOrigine == Node.NOIR) {
        supprimerFixup(x);              // re-equilibrage si nécessaire
    }
    return true;
}
```

Complexité : $O(\log n)$ — recherche du nœud, du successeur éventuel, et correction, toutes bornées par la hauteur de l'arbre.

6. Recherche et parcours

6.1 Recherche

La recherche exploite la propriété d'ordre de l'ABR : à chaque nœud, on compare la clé recherchée à la clé du nœud courant et on descend à gauche ou à droite en conséquence. Grâce à l'équilibrage Rouge-Noir, cette recherche s'effectue toujours en $O(\log n)$, y compris dans le pire des cas.

```
private Node rechercherNode(Node x, String key) {
    while (x != NIL) {
        int cmp = key.compareTo(x.key);
        if (cmp == 0) return x;
        x = cmp < 0 ? x.left : x.right;
    }
    return NIL;
}
```

6.2 Les trois parcours

Les trois parcours explorent l'intégralité de l'arbre en $O(n)$, mais dans un ordre différent selon le moment où la racine est visitée par rapport à ses sous-arbres :

Parcours	Ordre	Utilité
Prefixe	Racine -> Gauche -> Droite	Reconstituer l'arbre / le serialiser
Infixe	Gauche -> Racine -> Droite	Obtenir les clés triées par ordre croissant
Postfixe	Gauche -> Droite -> Racine	Supprimer/libérer l'arbre en toute sécurité

Exemple concret (parcours infixe sur l'arbre de test contenant des noms de fruits) : le parcours Gauche-Racine-Droite restitue exactement l'ordre alphabétique — *ananas*, *banane*, *citron*, *corossol*, *fraise*, ... — ce qui constitue une vérification visuelle immédiate de la correction de l'ABR.

7. Amelioration : validation automatique des proprietes

Au-dela de l'implementation de base, une methode `estValide()` a ete ajoutee pour verifier automatiquement, apres chaque operation, que les cinq proprietes Rouge-Noir sont respectees. Cette methode s'appuie sur un parcours recursif qui : verifie l'absence de deux noeuds ROUGE consecutifs, et calcule la hauteur noire de chaque sous-arbre en s'assurant qu'elle est identique a gauche et a droite.

```
private int verifierNoeud(Node x) {
    if (x == NIL) return 1;
    if (x.isRed() && (x.left.isRed() || x.right.isRed())) {
        return -1; // propriete 4 violee
    }
    int hg = verifierNoeud(x.left);
    int hd = verifierNoeud(x.right);
    if (hg == -1 || hd == -1 || hg != hd) {
        return -1; // propriete 5 violee
    }
    return hg + (x.color == Node.NOIR ? 1 : 0);
}
```

Cet outil de validation a permis de valider automatiquement l'implementation lors d'un **test de stress** de 5000 insertions puis 5000 suppressions effectuees dans un ordre aleatoire : l'arbre est reste valide apres *chaque* operation, et s'est correctement vide completement a la fin du test.

Resultat du test de stress

5000 insertions -> valide = true, taille = 5000, hauteur = 15

5000 suppressions -> vide = true, valide = true, taille = 0

Pour une hauteur theorique maximale de $2 \times \log_2(5001) \approx 24.4$, l'arbre obtenu a une hauteur reelle de seulement 15 — bien en-deca de la borne, ce qui confirme la qualite de l'equilibrage.

8. Amelioration : interface graphique JavaFX

Afin de visualiser concrètement le fonctionnement de l'arbre — et notamment de *voir* les rotations et recolorations se produire en direct — une application JavaFX (RedBlackTreeApp.java) a été développée. Elle dessine l'arbre à chaque opération, avec les couleurs réelles des nœuds (rouge/noir), et affiche en permanence le nombre de nœuds, la hauteur, et le résultat de `estValide()`.

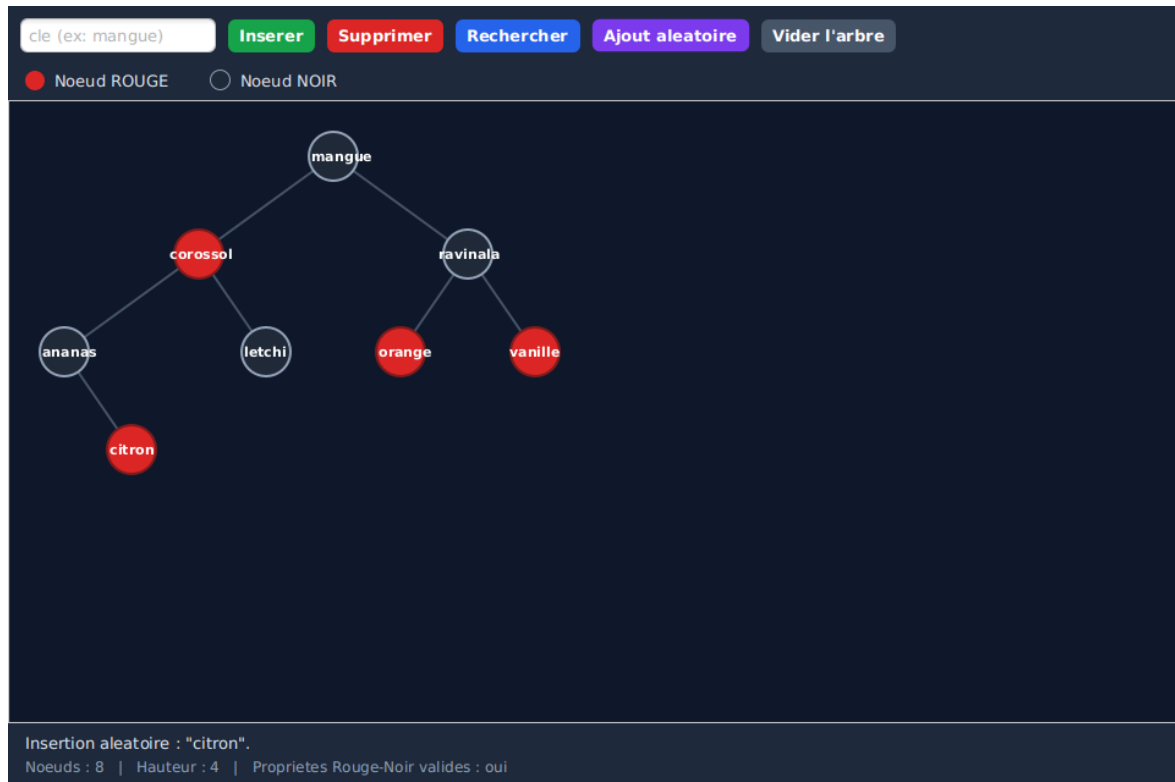


Fig. 1 — Arbre après plusieurs insertions : équilibrage automatique visible, couleurs rouge/noir affichées en direct.

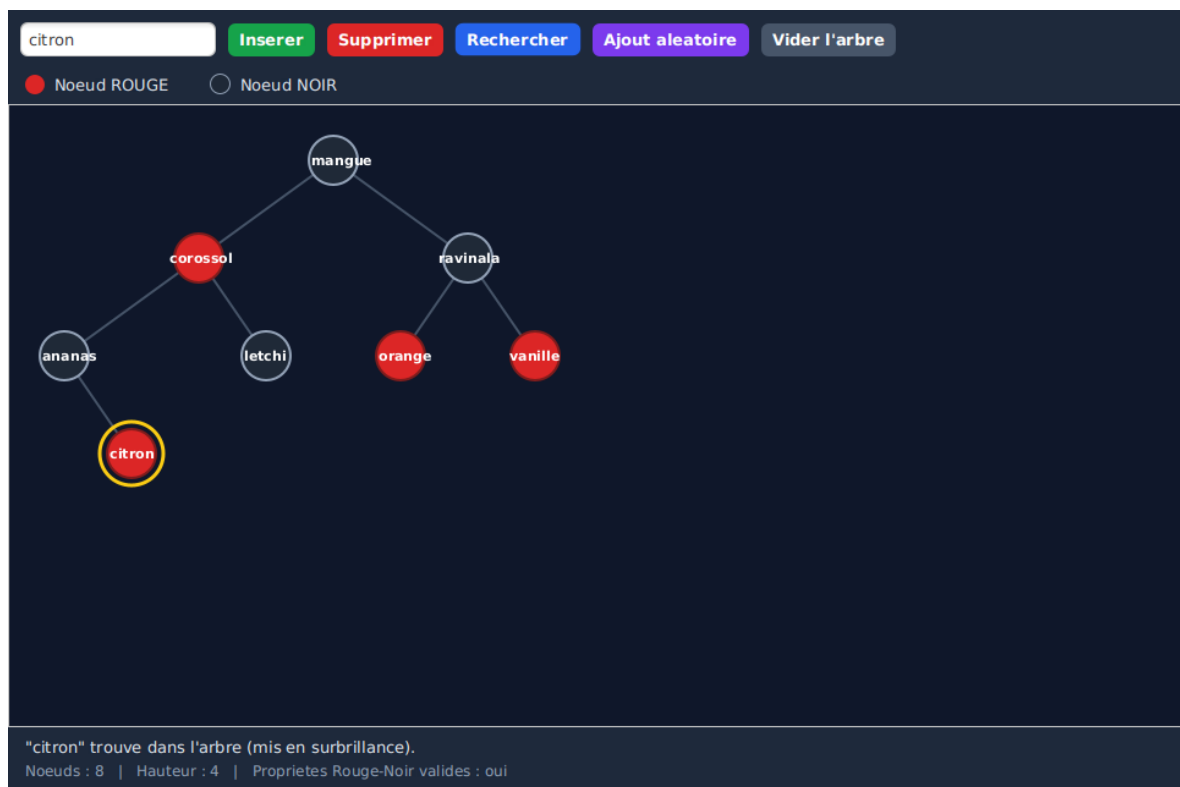


Fig. 2 — Fonction Recherche : le noeud trouve ("citron") est mis en surbrillance (halo jaune).

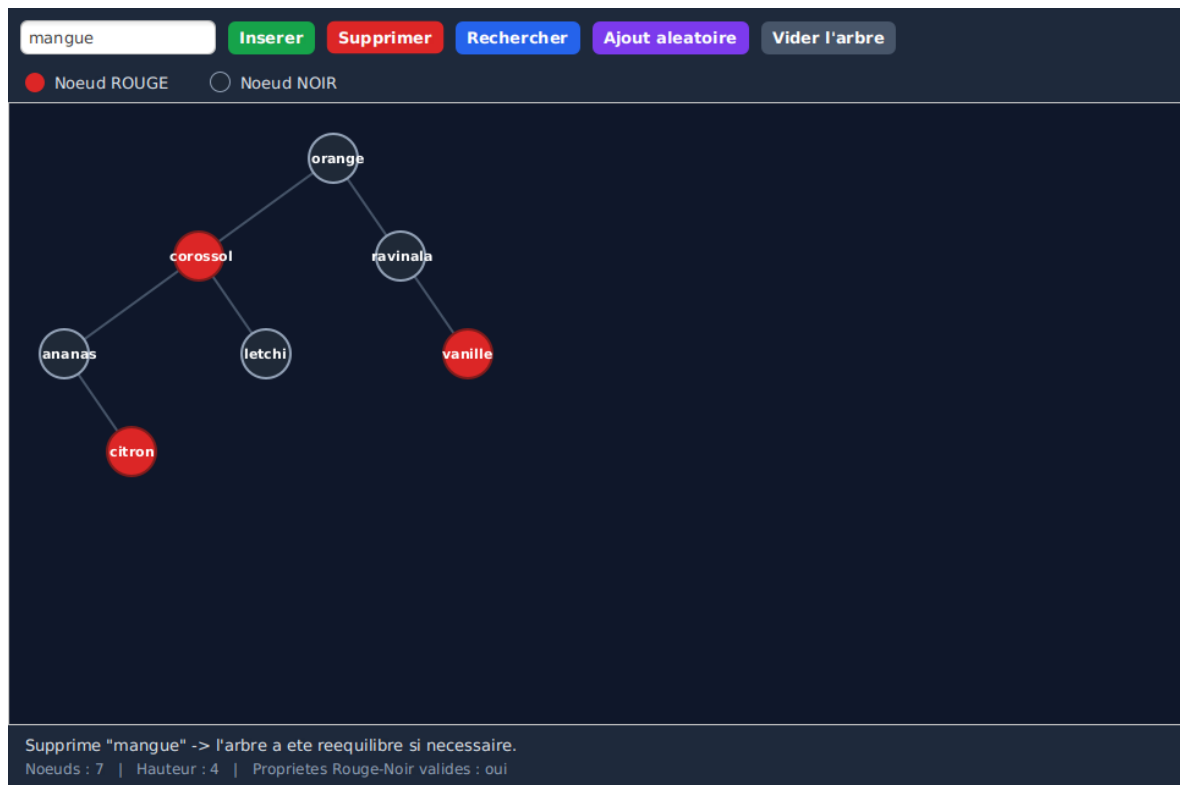


Fig. 3 — Après suppression de la racine ("mangue") : le successeur ("orange") la remplace et l'arbre reste valide (hauteur = 4, propriétés respectées).

8.1 Fonctionnalités de l'interface

- Insertion, suppression et recherche via un champ de texte et des boutons dédiés.
- Bouton "Ajout aléatoire" pour peupler rapidement l'arbre à des fins de démonstration.
- Mise en surbrillance du nœud trouvé lors d'une recherche.
- Recalcul et redessin automatique de la disposition des nœuds à chaque opération (positionnement basé sur un parcours infixe pour l'axe horizontal, et la profondeur pour l'axe vertical).
- Barre de statistiques en temps réel : nombre de nœuds, hauteur, validité des propriétés Rouge-Noir.

9. Recapitulatif des complexites

Operation	Complexite temporelle	Justification
Recherche	$O(\log n)$	Descente le long d'un seul chemin, de longueur bornee par la hauteur de l'arbre.
Insertion	$O(\log n)$	Insertion ABR classique + au plus $O(\log n)$ recolorations et 2 rotations.
Suppression	$O(\log n)$	Recherche + suppression ABR + au plus 3 rotations en correction.
Parcours (prefixe/infixe/postfixe)	$O(n)$	Chaque noeud est visite exactement une fois.
Validation (estValide)	$O(n)$	Parcours recursif complet de l'arbre.

A titre de comparaison, un ABR non equilibre presente les memes complexites en *moyenne*, mais degenerate a $O(n)$ dans le pire cas (insertion de cles deja trie'es). L'arbre Rouge-Noir garantit $O(\log n)$ dans **tous les cas**, ce qui justifie le surcout algorithmique (gestion des couleurs, rotations) introduit par sa maintenance.

10. Conclusion

Ce projet a permis d'implémenter, sans recourir à aucune bibliothèque de structure de données, un Arbre Binaire Rouge-Noir complet : insertion et suppression avec re-equilibrage automatique, recherche, et les trois parcours classiques (prefixe, infixe, postfixe). Deux améliorations significatives ont été apportées au-delà du strict nécessaire : une **validation automatique** des cinq propriétés Rouge-Noir, utilisée pour vérifier la correction de l'implémentation via un test de stress de 5000 opérations, et une **interface graphique JavaFX** permettant de visualiser l'équilibrage de l'arbre en temps réel.

Ce travail illustre concrètement l'intérêt des structures de données auto-équilibrées : la garantie d'une complexité $O(\log n)$ dans le pire cas, un atout essentiel dès que le volume de données ou l'imprévisibilité de l'ordre d'insertion devient significatif.